

Network Latency in Distributed LLM Inference

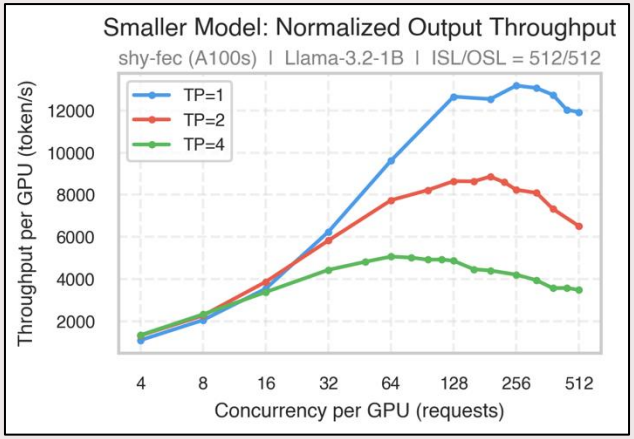
PROGRESS UPDATE: JUNE 3, 2026

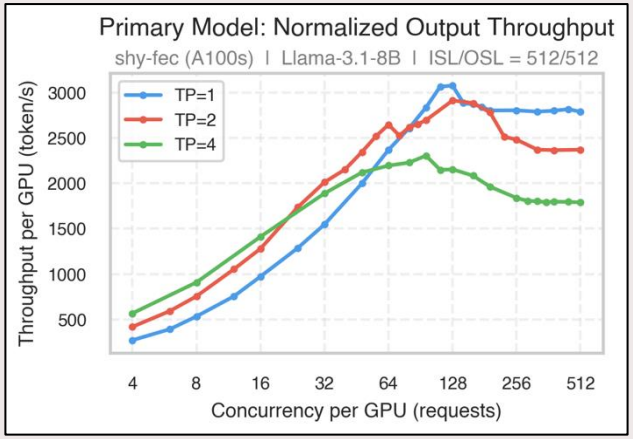
Kelsey Penners

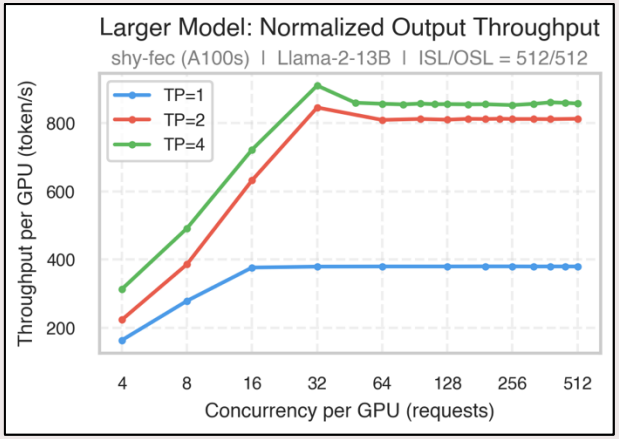
Status / Progress

- Reran 13B model at the consistent `gpu_mem_util = 0.85`.
- *Interconnect comparison:* tested baseline config with Peer-to-peer (P2P) via NVLink and forced fallback to PCIe via shared memory (SHM) by disabling P2P.
- *Hardware measurements*: combined `p2pBandwidthLatencyTest` and `nccl-tests` to model expected communication overhead versus observed overhead at runtime.
- *Power analysis:* measuring GPU power usage during benchmarking using `nvidia-smi`; working on getting the data in order to measure energy efficiency

TP Scaling with Model Sizes: shy-fec



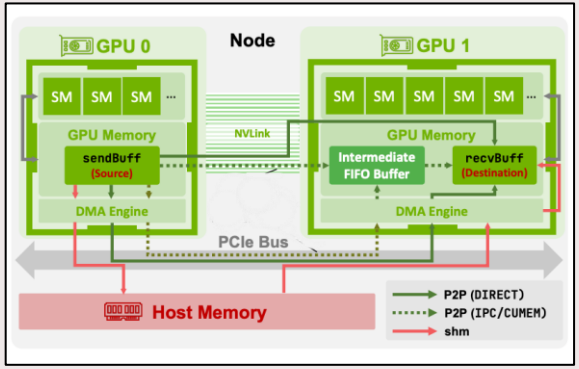




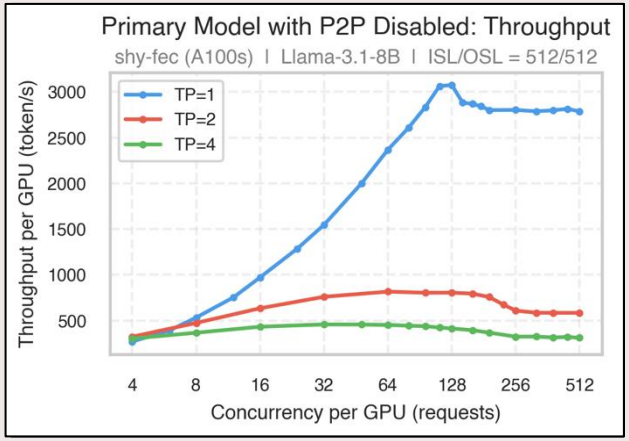
- Small models don’t benefit from TP due to communication overhead
- With NVLink, TP=4 in optimal for 13B -> memory and compute benefits outweigh communication cost
- How well an LLM scales relies directly on the size of the model and available VRAM

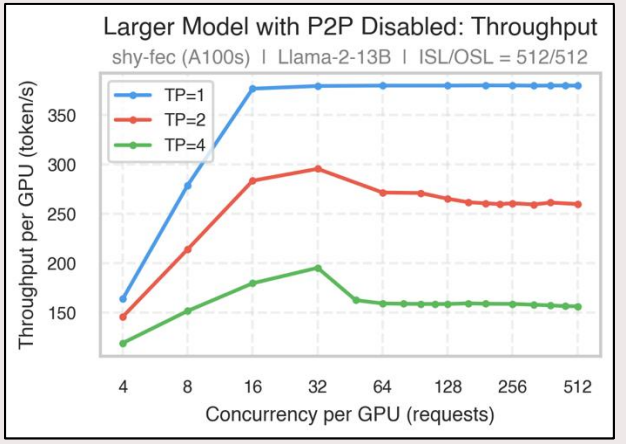
TU/e

TP Scaling: Peer-to-peer (P2P) disabled



Source: [Demystifying NCCL Paper]





- Same hardware, same model, different interconnect transport
- Scaling collapses throughput on less performative interconnects, throughput drops ~4×
- Unlike the case on NVLink, distributing the larger model becomes disadvantageous on PCIe
- Interconnect quality determines if distributing LLM inference is, not model size alone.

Hardware Measurement Methods

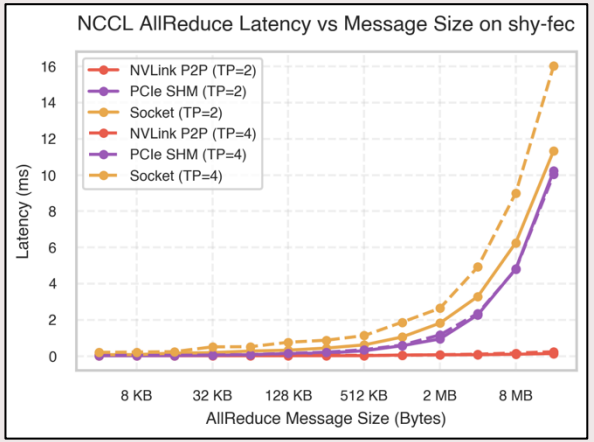
- `p2pBandwidthLatencyTest`:
  - measures GPU-to-GPU data transfer latency and bandwidth
  - direct look at hardware layer
  - latency test runs buffer copies of 1 integer, size 4 bytes
    bandwidth test copies 10 million integers, 40 MB
- `nccl-tests`:
  - measures collective communication (e.g., AllReduce latency)
  - reveals communication cost under tensor parallelism
  - will use to predict communication overhead
    - 2 × num layers = total AllReduce operations per prefill / decode step

P2P=Enabled Latency (P2P Writes) Matrix (us)

GPU	0	1	2	3	4	5	6	7
0	3.32	3.46	3.49	3.56	3.56	3.53	3.53	3.46
1	3.56	2.94	3.53	3.50	3.50	3.49	3.55	3.50
2	3.53	3.57	3.26	3.52	3.45	3.51	3.46	3.46
3	3.57	3.53	3.46	3.30	3.54	3.52	3.50	3.50
4	3.58	3.64	3.61	3.63	3.14	3.62	3.67	3.60
5	3.65	3.64	3.63	3.56	3.61	3.11	3.64	3.64
6	3.03	3.03	3.10	3.01	3.08	3.06	2.65	3.09
7	3.06	3.10	3.03	3.02	3.10	3.14	3.09	2.44

P2P=Disabled Latency Matrix (us)

GPU	0	1	2	3	4	5	6	7
0	3.31	25.54	24.79	24.59	25.48	25.54	24.67	26.55
1	25.60	2.95	24.60	24.61	25.48	25.49	24.67	24.68
2	25.52	24.59	3.28	24.60	25.51	25.46	24.62	24.59
3	24.61	24.59	24.66	3.29	25.47	25.18	24.60	24.60
4	24.68	24.61	24.71	24.68	3.13	25.55	26.57	24.70
5	24.70	24.72	24.78	24.69	25.59	3.11	24.92	24.72
6	24.69	24.62	24.60	24.60	25.54	25.35	2.65	24.69
7	24.74	24.59	24.68	24.59	25.20	25.46	24.66	2.42

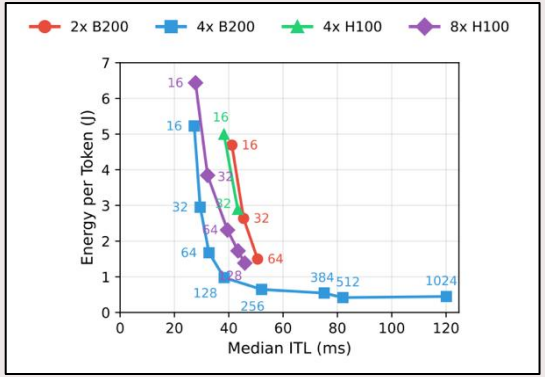


Energy / Power Profiling

- *Approach:* query `nvidia-smi` during benchmark runs to log power consumption

$$E_token = \sum\{P_i \times dt\} / N_tokens$$

- Compute joules per token from power consumption to show how power efficiency is impacted across TP degrees and interconnect selections.
- Data being collected, working on processing and visualizing it!
- Batch size is main knob for inference energy cost according to related work.



Source: [Where Do the Joules Go? Paper]

Discussion

- Disabling custom AllReduce in vLLM to allow for more direct comparison
- SLO Analysis: can depend on mean, median, or tail values?

Next Steps

- Process power draw data to find J/token across configs
- Mark SLO boundaries, relate all configs to one objective
- Continue with relating hardware measurements to observed runtime data
- Writing!

Thank you!